

面向大规模查询的数据结构算法

上海市育才中学 丁梓洋

摘 要

在信息学奥林匹克竞赛（NOI）及相关竞赛中，数据结构题型常以高效查询与优化计算为核心，考察选手对算法设计与实现的能力。其中，离线算法因其能够高效处理大规模查询问题，在竞赛中得到了广泛应用。本文以「求区间内不同元素个数」问题为例，分析了莫队算法、CDQ 分治、分块等典型离线算法，探讨了它们在竞赛环境下的适用性、复杂度及优化技巧。通过对不同算法的深入比较，本文为竞赛选手提供了一套较为系统的方法论，以帮助读者理解数据结构题的核心思想，提高算法设计与优化能力。

在实际工程应用中，大规模数据查询广泛存在于数据库系统、搜索引擎、数据挖掘等领域。传统的暴力查询方法在数据量较大时难以满足性能需求，因此需要利用高效的数据结构。本文亦为实际工程中高效处理大规模查询的问题提供理论支撑和实践指导。

关键词 信息学奥赛 离线算法 数据结构 莫队算法 分治思想 分块 树状数组 线段树 归并树 扫描线

目 录

第一章 例题	1
1.1 题目大意	1
1.2 问题分析	1
第二章 分块	1
2.1 思路分析	1
2.2 实现	1
2.3 时间复杂度	2
2.4 衍生题目	3
第三章 普通莫队算法	3
3.1 思路分析	3
3.2 实现	4
3.3 时间复杂度	4
3.4 优化	5
3.4.1 最优块的大小	5
3.4.2 奇偶块交错排序优化	6
3.4.3 数据离散化	6
3.4.4 优化 <i>cnt</i> 数组	6
3.5 衍生题目	6
第四章 树状数组 / 线段树	7
4.1 思路分析	7
4.2 实现	7
4.3 时间复杂度	8
第五章 归并树	8
5.1 思路分析	8
5.2 实现	8
5.3 时间复杂度	9
5.4 优化	9
第六章 扫描线	9

6.1 思路分析	9
6.2 实现	9
6.3 时间复杂度	11
第七章 总结	11
参考文献	13
致 谢	13

第一章 例题

1.1 题目大意

给定一个长为 N ($1 \leq N \leq 10^6$) 的序列 $\{a_i\}$ ($1 \leq a_i \leq 10^6$)，给定指令： $Q\ l\ r$ ，询问区间 $[l, r]$ 内有多少个不同的数。

现有 M ($1 \leq M \leq 10^6$) 条指令。要求对每条询问指令输出其答案。内存限制 512 MB，时间限制 2.0 s。¹

1.2 问题分析

该问题属于典型的区间查询问题，要求高效地处理大量查询，序列长度和询问次数都在 10^6 数量级。一种暴力的思路是对每个询问单独统计不同元素的个数，但这在最坏情况下时间复杂度会达到 $O(NM)$ ，显然无法满足时间限制。

由于题目明确是静态查询，也就是说输入的序列不会发生变化，我们可以设计一个离线算法来处理所有询问。

第二章 分块

2.1 思路分析

分块算法的一般思想如下：首先，我们需要将序列分为若干个长度相等的区间，称之为“块”。在计算询问之前，先预处理每个块内和连续块内的不同元素的个数，存储到一个二维数组中。对于一个询问区间 $[l, r]$ ，如果 l 和 r 在同一个块内，则直接遍历区间内的每一个元素，暴力求解；如果 l 和 r 不在同一个块内，利用 $[l, r]$ 真包含的连续块的预处理结果，避免重复计算，减少时间复杂度。

2.2 实现

首先，我们需要预处理 a_i 在序列中上一次出现的位置 bef_i 和下一次出现的位置 $next_i$ 。一种简单的方案可以随着依次读入序列 $\{a_i\}$ 的过程进行预处理。定义一个哈希表 $last_x$ ，表示值 $x = a_i$ 在当前读入的数列 $[a_1, a_i]$ 中，最后出现的位置。那么即可赋

¹来源：洛谷 P1972 [SDOI2009] HH 的项链，有修改

值 $bef_i \leftarrow last_x$, $nxt_{last_x} \leftarrow i$, $last_x = i$ 。最终随着序列 a_i 的读入, 所有数据的 bef 与 nxt 值均处理完成。当然, 我们也可以在读完序列之后, 利用哈希表 $last$ 对序列作正向和反向的遍历, 分别计算 bef 和 nxt 数组。两种预处理方案的时间复杂度均为 $O(N)$ 。

然后, 我们需要进行分块的预处理操作。先将序列分为若干个块。设序列长度为 N , 令块的大小 $S = \lfloor \sqrt{N} \rfloor$, 则一共会有 $D = \lceil \frac{N}{S} \rceil$ 个块。对于每个块, 暴力地计算块内不同元素的个数。此操作可以借用一个布尔数组 exi_{a_i} 记录 a_i 是否出现过。具体来说, 若处理的块为 $[a_m, a_n]$, 从 a_m 开始遍历块内的每个元素。如果 $exi_{a_i} = 0$, 则说明 a_i 第一次在块内出现, 则块内不同元素的个数 $cnt \leftarrow cnt + 1$; 且令 $exi_{a_i} = 1$, 标记值 a_i 已在块内出现。另外, 我们也可以借助 bef 数组来计算。如果 $bef_i < m$, 即上一个值 a_i 出现在第 m 个元素之前, 也就是说从 m 到 $i - 1$ 内都没有出现过值 a_i , a_i 在块内第一次出现, 令 $cnt \leftarrow cnt + 1$ 。

记二维数组 $f_{i,j}$ 表示第 i 到第 j 个块内不同元素的个数, 则第 i 个块内不同元素的个数即为 $f_{i,i}$ 。

我们可以使用递推的方法, 计算 f 数组的所有值。其递推公式为 $f_{i,j} = f_{i,j-1} + \Delta cnt$, 其中 Δcnt 表示第 j 个块内相对第 $i \sim j - 1$ 个块新增的不同元素的个数, 即仅在第 j 个块内出现, 而没有在第 i 到第 $j - 1$ 个块内出现的元素的个数。此操作可利用 bef 数组实现。对于每个递推过程, 我们可以在 $O(S)$ 的时间复杂度内完成转移。至此, 我们完成了所有预处理操作。

对于每条询问 $[l, r]$, 先判断 l 与 r 是否在同一个块内。如果在同一个块内, 说明该询问区间的长度小于等于 S , 可以直接暴力计算, 其时间复杂度为 $O(S)$; 如果不在同一个块内, 我们可以利用 f 数组求解。设 l 所在的块为 b_l , r 所在的块为 b_r , 则 $[l, r]$ 内的不同元素的个数即为 f_{b_l+1, b_r-1} 加上 b_l 和 b_r 内的不同元素的个数。对于 b_l 和 b_r 内的不同元素的个数, 由于两者需要计算的区间长度均小于等于 S , 可以直接暴力计算。此操作的时间复杂度也为 $O(S)$ 。这步操作正是体现了分块优化时间复杂度的核心思想。

2.3 时间复杂度

该算法的时间复杂度为 $O((N + M)\sqrt{N}) = O(N\sqrt{N})$, 其中 N 为序列长度, M 为询问区间的数量。预处理 bef 和 nxt 数组的时间复杂度为 $O(N)$; 预处理 f 数组的时

间复杂度为 $O(DS^2)$ ，即 $O(N\sqrt{N})$ ；对于所有查询，时间复杂度为 $O(MS)$ ，即 $O(M\sqrt{N})$ 。

2.4 衍生题目

分块是一种灵活且通用的思想。即思考过程相对简单，代码实现逻辑单一明白。尤其在处理一些复杂的查询问题时，能充分利用其灵活性，处理那些普通树状数组或线段树难以做到的情况。对于区间查询和更新问题，分块可以在不需要过度复杂结构的情况下，提供较为高效的解决方案。

分块的一个关键优势在于它能够将问题的复杂度从一个大规模的结构中分解到若干小块中，这样能够更好地适应实际问题中的各种复杂约束。另外，分块不仅限于区间问题，还可以扩展到其他类型的问题，比如求最小公倍数、最大公约数、处理频率分布等。

分块的实现思路和其他数据结构不同，时间复杂度也不同。在应对不同题目时，读者可以根据自己擅长的解法、评估时间复杂度等因素，选择合适的方法。

以下推荐几道不同难度的题目，均可同时使用分块和数据结构完成，可供读者练习和比较两者的思路：

- 洛谷 P4145 上帝造题的七分钟 2 / 花神游历各国
- 洛谷 P1471 方差
- 洛谷 P1975 [国家集训队] 排队

对于难度更高的大分块题目，推荐：洛谷题单 - YNOI 大分块系列²。

第三章 普通莫队算法

3.1 思路分析

莫队算法是一种经典的离线算法，适用于多次的区间查询的问题。它可以利用上一个查询区间的结果，减少重复计算。

假设已经计算好了区间 $[l, r]$ 内不同元素的个数为 cnt ，现询问区间 $[l, r + 1]$ 内的不同元素的个数。如果区间 $[l, r]$ 内没有 a_{r+1} ，则答案即为 $cnt + 1$ ，反之为 cnt 。类似地，我们可以用这种方法从一个已经计算好的区间 $[l_i, r_i]$ 转移到 $[l_{i+1}, r_{i+1}]$ 。

²洛谷题单 - YNOI 大分块系列：<https://www.luogu.com.cn/training/44148>

但是，使用上述思路暴力地转移每个区间的最坏时间复杂度仍为 $O(MN)$ 。莫队算法通过对询问离线，将询问按照更优的顺序排序，从而减少总转移距离，降低时间复杂度。

3.2 实现

先进行离线操作，即对询问区间进行排序。莫队算法也使用了类似分块的思想，将区间分为多个块，所有左端点 l 在同一个块内的区间为一个整体。对于每个块，按照区间的右端点从小到大进行排序。最终的结果如图 1 所示。这样离线便是莫队算法的核心，其好处在后文可以体现。

用两个指针 p, q 表示现已计算好的区间 $[p, q]$ ，记该区间内不同元素的个数为 cnt ，区间内数 x 出现的次数为 sum_x 。在处理区间 $[l_i, r_i]$ 时，将左指针 p 逐位移动到 l_i 。如果 p 需要向左移动 1 位，且 $sum_{p-1} = 0$ ，即 a_{p-1} 在 $[p, q]$ 中没有出现过，则将 cnt 增加 1；否则 cnt 保持不变。同时，也要将 $sum_{a_{p-1}}$ 增加 1，表示新的 $[p, q]$ 内该数的出现次数增加 1 次。如果 p 需要向右移动 1 位，则将 $sum_{a_{p+1}}$ 减少 1。如果现在 $sum_{a_{p+1}} = 0$ ，即最后的一个 a_{p+1} 被移出了 $[p, q]$ ，则将 cnt 减少 1；否则 cnt 保持不变。右端点 q 的移动方式与左端点相反。

通过这样的转移方式，我们可以在上一个区间的答案的基础上计算得到下一个区间的答案。经过离线排序后的询问区间，如图 1 所示，我们不难发现：左指针在同一个块内的移动距离较小，而右指针在每个块内都是单调增加的。这样的离线使总转移距离尽可能小，这便是莫队算法的核心思想。

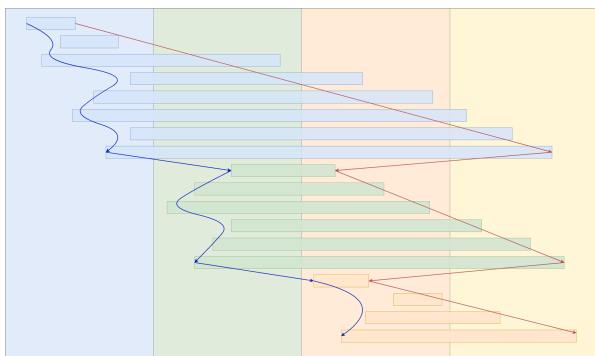


图 1 普通莫队算法

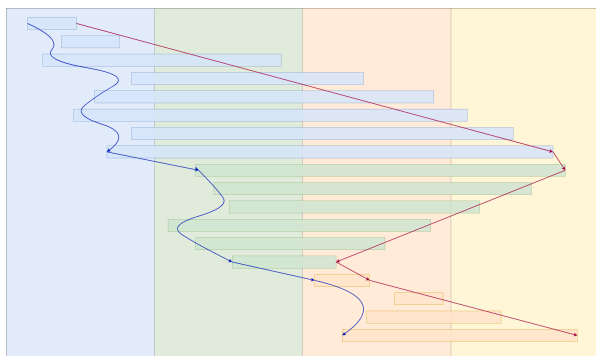


图 2 奇偶块交错排序优化的莫队算法

3.3 时间复杂度

在 N 与 M 同阶时，莫队算法的时间复杂度为 $O(N\sqrt{N})$ 。其中，排序的时间复杂度为 $O(N \log N)$ ；区间之间转移的总时间复杂度为 $O(N\sqrt{N})$ 。

其中，对于区间之间转移的时间复杂度，我们可以通过以下方法推得：

令每一块中 l 的最大值为 $max_1, max_2, max_3, \dots, max_{\lceil \sqrt{N} \rceil}$ 。由第一次排序可知， $max_1 \leq max_2 \leq \dots \leq max_{\lceil \sqrt{N} \rceil}$ 。显然，对于每一块暴力求出第一个询问的时间复杂度为 $O(N)$ 。考虑最坏的情况，在每一块中， r 的最大值均为 N ，每次修改操作均要将 l 由 max_{i-1} 修改至 max_i 或由 max_i 修改至 max_{i-1} 。考虑 r ：因为 r 在块中已经排好序，所以在同一块修改完它的时间复杂度为 $O(N)$ 。对于所有块就是 $O(N\sqrt{N})$ 。重点分析 l ：因为每一次改变的时间复杂度都是 $O(max_i - max_{i-1})$ 的，所以在同一块中时间复杂度为 $O(\sqrt{N} \cdot (max_i - max_{i-1}))$ 。

将每一块 l 的时间复杂度合在一起，通过裂项求和可得：

$$\begin{aligned} O(l) &= O\left(\sqrt{N}(max_1 - 1) + \sqrt{N}(max_2 - max_1) + \dots + \sqrt{N}(max_{\lceil \sqrt{N} \rceil} - max_{\lceil \sqrt{N} \rceil - 1})\right) \\ &= O\left(\sqrt{N} \cdot (max_1 - 1 + max_2 - max_1 + \dots + max_{\lceil \sqrt{N} \rceil} - max_{\lceil \sqrt{N} \rceil - 1})\right) \\ &= O\left(\sqrt{N} \cdot max_{\lceil \sqrt{N} \rceil - 1}\right) \end{aligned}$$

又因为 $max_{\lceil \sqrt{N} \rceil - 1}$ 最大为 N ，因此 l 的总时间复杂度最坏情况下为 $O(N\sqrt{N})$ 。^[1]

3.4 优化

该题的数据使用莫队算法可能会被卡常，上述思路在洛谷的测试结果仅为 22 分。以下是几种常见且重要的优化方法。

3.4.1 最优块的大小

与分块有关的算法，块的大小对常数时间复杂度有很大的影响。设块长度为 S ，那么对于任意多个在同一块内的询问，指针的移动的距离为 N ，共有 $\frac{N}{S}$ 个块，移动的总距离就是 $\frac{N^2}{S}$ 。由于移动可能跨越块，所以还要加上 $O(MS)$ ，总复杂度为 $O(\frac{N^2}{S} + MS)$ 。当 S 取 $\frac{N}{\sqrt{M}}$ 时时间复杂度最优，为 $O(N\sqrt{M})$ 。

块的大小对莫队算法的时间复杂度影响非常显著。例如， M 与 $\sqrt{N}$ 同阶时，最优的块大小 $S = \frac{N}{\sqrt{M}} = N^{1-\frac{1}{2} \times \frac{1}{2}} = N^{\frac{3}{4}}$ ，时间复杂度可以达到 $O(N^{\frac{5}{4}})$ 。如果块的大小取 $\sqrt{N}$ ，其时间复杂度只有 $O(N\sqrt{N})$ 。^[1]

在本题中， N 与 M 同阶，因此块的大小取 $\sqrt{N}$ 即可。

3.4.2 奇偶块交错排序优化

同一个块内的询问区间的右端点 r 均是递增的。在两个块之间转移时，指针 q 的移动距离较远。因此，我们可以做如下优化：对于编号为奇数的块，块内的询问区间按照 r 递增排序；对于编号为偶数的块，块内的询问区间按照 r 递减排序。这样，指针 q 在两个块之间的移动距离将会减小，可以在一定程度上优化常数时间复杂度，见图 1 与图 2。使用该优化，在洛谷的测试结果可以提高 8 分。

3.4.3 数据离散化

由于该题输入数据 $a_i \in [1, 10^6]$ ，范围较大。因此，我们可以考虑将输入序列的值进行离散化。经过离散化后，对 cnt 数组的访问和修改更加连续，每次 p, q 指针的转移速度更快。由于莫队算法的瓶颈在于 p, q 指针的转移，因此经过离散化后莫队算法的常数时间复杂度得以有效减少。使用该优化，在洛谷的测试结果可以提高 48 分。^[2]

3.4.4 优化 cnt 数组

通过上一种优化方法，我们可以发现， cnt 数组的访问和修改较为耗时。实际上，我们可以不记录 cnt 数组。考虑在读入时预处理 a_i 上一次出现的位置 bef_i 和下一次出现的位置 $next_i$ 。在转移指针 p, q 时，只需要访问 bef_i 或 $next_i$ 中的一个元素便可知道数 a_i 是否在原区间 $[p, q]$ 出现过，且无需进行对 bef 或 $next$ 的修改操作。另外，在使用此方法优化时，离散化输入序列的值就没有必要了。使用该优化，在洛谷测试结果可以提高 60 分，顺利通过该题，最大的数据甚至可以在 $1.01s$ 内通过。^[3]

3.5 衍生题目

本题的数据对莫队算法进行了一定的卡常，不适合作为莫队算法的入门练习。但近年来，不少省选题可以通过一定的优化，使用莫队算法解决。相比正解，莫队算法的思路和实现较为简便。因此，本题可作为莫队算法常数优化的一道良好的练习题。

以下提供几道普通莫队算法的模板题，可供读者快速掌握莫队算法，更好地理解莫队算法的核心思想：

- 洛谷 P2709 小 B 的询问
- 洛谷 P1494 [国家集训队] 小 Z 的袜子
- 洛谷 P4462 [CQOI2018] 异或序列

第四章 树状数组 / 线段树

4.1 思路分析

使用树状数组或线段树这类数据结构可以高效地处理区间查询和单点更新。

对于每条区间询问，我们可以在 $O(\log N)$ 的时间复杂度内求出区间内所有元素的和、最大值、最小值等信息。令每个数的贡献值都为 1，使用区间求和操作可以等价于求区间内所有元素的个数，但是对于值相同的元素会有重复计算。如果区间内出现多个相同的数，其中只有一个数的贡献值为 1，其他数的贡献值均为 0，我们便可以通过区间求和得出区间内不同元素的个数。

4.2 实现

使用这类数据结构解决时，我们也需要考虑将询问离线。具体来说，将每个查询区间按照其右端点 r 升序排序。这样的离线方便维护上文所述的 01 树状数组。记排序后第 i 个询问为 $[l_i, r_i]$ 。

使用树状数组维护每个位置的状态。每次根据 r 的升序计算询问区间的同时，更新树状数组。假设上一个处理的区间为 $[l_i, r_i]$ ，则现在处理的区间为 $[l_{i+1}, r_{i+1}]$ ，我们需要更新树状数组 $[c_{r_i}, c_{r_{i+1}}]$ 区间的每个值。

对于相同的值，我们只关心这个值在区间中出现的最右一个。如果 a_i 在 $[1, i)$ 之间没有出现过（即 $bef_i = 0$ ），则将 c_i 记为 1；如果 a_i 在 $[1, i)$ 之间出现过（即 $bef_i \neq 0$ ），则将 c_i 记为 1 的同时将 c_{bef_i} 记为 0（在维护树状数组时，通过将 c_i 加 1 或减 1 的方式实现）。因此，我们就避免了重复计算同一个数值。 $[l_i, r_i]$ 区间内的不同数的个数即为 $\sum_{j=l_i}^{r_i} c_j$ （树状数组通过 $query(r_i) - query(l_i - 1)$ 实现）。

除了树状数组，我们还可以使用线段树来实现，实现方法与树状数组类似。线段树的每个节点维护区间内的不同数的个数。在更新和查询线段树时，我们可以通过递归地更新或查询左右子树来实现。使用线段树解决相对树状数组来说，常数时间复杂度较树状数组更大，且代码较为复杂，在本题没有明显优势。

4.3 时间复杂度

该算法的时间复杂度为 $O((N + M) \log N) = O(N \log N)$ ，其中 N 为序列长度， M 为询问区间的数量。预处理 $\{bef_i\}$ 的时间复杂度为 $O(N)$ 。将查询按照右端点 r 排序的时间复杂度为 $O(M \log M)$ 。对于所有查询，更新树状数组或线段树的时间复杂度为 $O((N + M) \log N)$ 。

使用树状数组 / 线段树实现本题游刃有余，不需要过多的优化，其关键在于如何对询问离线，以及树状数组 / 线段树维护什么内容。与莫队算法类似，其核心在于对询问进行离线处理。

第五章 归并树

5.1 思路分析

归并树算法通过构建归并树维护序列的辅助信息，并结合二分查找快速回答每次询问。它结合了归并排序和线段树的思想。在构建时，它递归地将序列划分为左右子区间，对每个子区间进行排序，并将有序序列存储在对应的线段树节点上。对于询问 $[l, r]$ ，我们可以通过线段树的区间分解，将其拆分为 $O(\log N)$ 个节点区间，每个节点存储的 bef 值是有序的。利用二分查找，我们可以在 $O(\log N)$ 时间内统计每个节点中 $bef_i < l$ 的元素个数，从而计算答案。

5.2 实现

在读入序列 $\{a_i\}$ 时预处理每个元素 a_i 在序列中上一次出现的位置 bef_i 。

利用归并树来组织 bef 数组的数据。归并树是一棵线段树，每个节点负责一段区间，存储该区间内 bef 值的有序序列。具体实现时，从根节点开始递归划分区间。对于当前节点负责的区间 $[l, r]$ ，如果 $l = r$ ，说明是叶子节点，直接将 $bef[l]$ 存入该节点的数组中。如果 $l < r$ ，则将区间分为 $[l, mid]$ 和 $[mid + 1, r]$ 两部分，分别递归构建左子树和右子树。子树构建完成后，将左右子树的有序 bef 序列归并到当前节点的序列中。归并的过程类似于归并排序，使用双指针依次比较并合并，确保当前节点存储的 bef 序列也是有序的。由于每个元素只会被归并 $O(\log N)$ 次，建树总时间复杂度为 $O(N \log N)$ 。

对于每次查询，给定区间 $[l, r]$ ，需要在归并树上计算答案。查询函数从根节点开始，判断当前节点负责的区间 $[x, y]$ 与查询区间 $[l, r]$ 的关系。如果 $[x, y]$ 完全被 $[l, r]$ 包含，则直接在该节点的有序 bef 序列中使用二分查找，统计小于 l 的 bef 值的个数，这些位置对应的元素在 $[l, r]$ 内是首次出现。如果 $[x, y]$ 与 $[l, r]$ 部分重叠，则递归查询左子树（如果 $l \leq mid$ ）和右子树（如果 $mid < r$ ），将两部分的结果相加。

5.3 时间复杂度

归并树算法的时间复杂度为 $O((N + M) \log^2 N) = O(N \log^2 N)$ ，其中 N 为序列长度， M 为询问区间的数量。构建归并树的时间复杂度为 $O(N \log N)$ ；对于每个询问，我们需要在 $O(\log N)$ 个节点上进行二分查找，每个节点内二分的时间复杂度为 $O(\log N)$ ，因此对于单个询问的时间复杂度为 $O(M \log^2 N)$ 。

5.4 优化

朴素的实现仍需在 $O(\log N)$ 个节点上进行二分查找，总复杂度为 $O(M \log^2 N)$ ，可能面临常数过大的风险，在洛谷的测试结果为 88 分。考虑应用离线处理和优化技巧：将所有询问按左端点 l 递增排序，并在每个节点记录上一次二分查找的结果 d 。由于 l 单调递增，下次查询时可从 $d + 1$ 开始二分，缩小查找范围，从而降低平均时间复杂度。通过这样的优化，可以有效提高性能。

第六章 扫描线

6.1 思路分析

扫描线不仅可以解决二维矩形的面积并、周长并等问题，还可以解决二维数点问题。如查询区间中值在 $[x, y]$ 内的元素个数。将该问题映射到二维坐标系中，通过扫描线的思想，结合树状数组 / 线段树即可实现。对于本题，可采用类似的思路，其关键在于如何将题目转化为二维数点问题。

6.2 实现

在一个区间 $[l, r]$ 中，如果有多个相同的数，此时我们考虑在 $[l, r]$ 中第一次出现的元素，其余不产生贡献。那么一个数是否产生贡献可以通过 bef 数组求出。若 $bef_i <$

l ，则表明 a_i 在 $[l, r]$ 中第一次出现，产生贡献；否则不产生贡献。因此，该区间中不同元素的个数即为 $\sum_{i=l}^r [bef_i < l]$ 。

把 bef 数组映射到一个二维平面中，其中 bef_i 为点 (i, bef_i) 。至此，我们把原问题转化为：求二维平面中，以 $(l, 0)$ 为左下角、 $(r, l-1)$ 为右上角的矩形中的点的个数。参考图 3，左侧表示的是输入的序列 $\{a\}$ 以及预处理的数组 $\{bef\}$ ，询问的区间为 $[6, 12]$ ，我们可以把 bef 数组如图映射。绿色矩形 **ans** 内点的数量就是该区间内不同元素的数量，而红色矩形 **now** 表示的是在该区间内出现次数超过一次的元素。

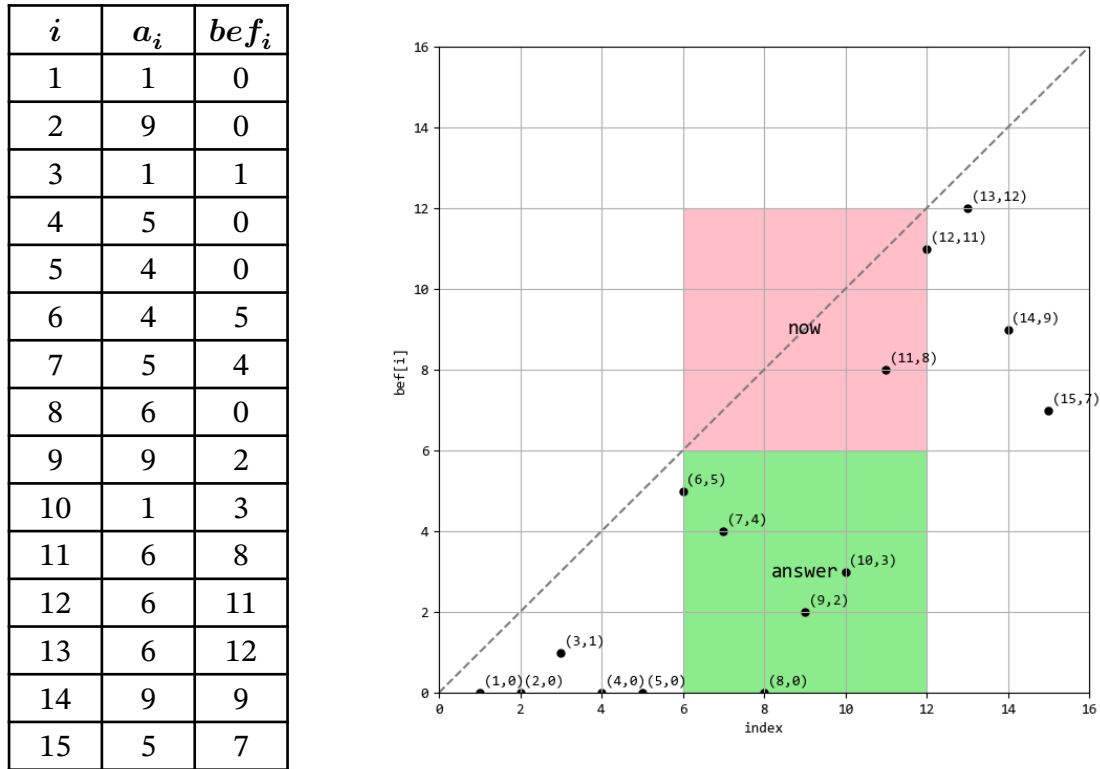


图 3 转化为二维数点问题

求该矩形内的点的数量可以通过差分的方式实现。由于该矩形左下角顶点的纵坐标一定为 0，矩形 $(l, 0) \rightarrow (r, l-1)$ 内点的数量即为矩形 $(0, 0) \rightarrow (r, l-1)$ 内的点的数量减去矩形 $(0, 0) \rightarrow (l-1, l-1)$ 内的点的数量。这样，我们便可以通过扫描线分别求出这两个差分矩形内的点的数量。

扫描线将直线 $x = 0$ 从左向右进行扫描，扫描的过程中用树状数组 c 维护纵坐标对应范围内点的数量。树状数组共有 2 种操作：单点增加和区间求和。对于每个元素 a_i ，需要插入点 (i, bef_i) 。因此，当扫描到 $x = i$ 时， c_{bef_i} 增加 1。对于每个询问区间 $[l_i, r_i]$ ，当扫描到 $x = l-1$ 时，查询 $\sum_{i=0}^{l-1} c_i$ 的值；当扫描到 $x = r$ 时，查询 $\sum_{i=0}^{l-1} c_i$

的值。两次查询的差值即为 $[l_i, r_i]$ 区间内不同元素的个数。使用线段树操作类似树状数组，常数略大。

6.3 时间复杂度

该算法的时间复杂度为 $O((N + M) \log(N + M)) = O(N \log N)$ 。预处理 *bef* 数组时间复杂度 $O(N)$ ；将询问操作和加点操作排序时间复杂度 $O((2M + N) \log(2M + N))$ ，每次树状数组 / 线段树的修改和查询操作的时间复杂度均为 $O(\log N)$ ，共有 N 次加点操作和 $2M$ 次查询操作。

第七章 总结

本文围绕区间不同元素个数查询问题，系统分析了分块、普通莫队算法、树状数组/线段树、归并树和扫描线五种经典算法。通过对算法复杂度、实现难度、优化方案的深入分析，我们可以发现每种算法各有特点，适用于不同场景。从时间复杂度角度看，树状数组/线段树和扫描线的复杂度为 $O(N \log N)$ ，理论上效率最高；分块和莫队算法次之，为 $O(N\sqrt{N})$ ；归并树虽然复杂度达到 $O(N \log^2 N)$ ，但在某些特定场景下表现同样优异。空间复杂度方面，分块、莫队算法为 $O(N)$ ，而归并树和扫描线需要 $O(N \log N)$ 的额外空间。这些差异使得在不同的应用场景下，选择合适的算法至关重要。

在信息学奥赛中，这些算法是解决区间查询问题的基础工具。近年来，省选和 NOI 试题中频繁出现需要高效处理区间统计信息的问题。选手需根据题目特点和个人熟练度选择合适算法，权衡时间复杂度和实现难度。特别值得一提的是，这些算法思想可以迁移到树上问题和图论问题中，如树上莫队、树上分块等高级技巧，进一步扩展了应用范围。除基础的区间查询外，这些算法还可以处理带修改的区间查询、二维区间查询等更复杂的变形，通过组合多种数据结构，如莫队结合树状数组、分块结合线段树等方式，解决更复杂的问题。对于竞赛选手来说，掌握这些算法不仅是解题的工具，更是培养算法思维的重要途径。

在实际工程应用中，大规模数据查询问题广泛存在于数据库系统、搜索引擎、数据挖掘、流式计算等领域。现代数据库系统的索引设计和查询优化借鉴了分块和树状结构的思想，如 B+ 树索引和分区表技术；搜索引擎的倒排索引和文档检索系统利用

了分块排序和树状结构进行高效检索；大数据处理框架如 Hadoop 和 Spark 在数据分片和聚合计算中应用了分块思想；时序数据库在处理时间序列数据时，利用区间树数据结构提升查询效率；实时计算系统处理流式数据时，利用滑动窗口结合高效的树形数据结构进行增量计算。这些实际应用证明了本文所讨论算法的现实价值，也启示我们在算法设计中要兼顾理论复杂度和工程实用性。

随着数据规模的持续增长和应用场景的多样化，这些经典算法还在不断发展。在并行计算环境下，分块等算法天然适合并行优化，利用多核处理器或分布式系统提升性能；针对动态修改的序列，发展基于持久化数据结构的高效维护算法；结合机器学习技术，设计自适应数据结构，根据数据特性和查询模式自动选择最优结构；开发更加通用的算法框架，降低实现门槛，使这些高效算法能够广泛应用于工程实践。

通过本文的系统分析，我们不仅学习了解决具体问题的多种方法，更重要的是理解了不同算法设计思想的本质和适用条件。在算法选择时，需综合考虑数据规模、查询特点、时间要求、空间限制等多种因素。例如，对于小规模数据或对实时性要求较高的场景，可能倾向于选择实现简单的树状数组；对于大规模数据的离线处理，莫队算法或扫描线可能是更好的选择；而面对复杂多变的查询需求，分块算法的灵活性则显得尤为重要。无论在竞赛还是工程中，掌握多种算法工具，并能根据具体问题特点进行有针对性的选择和优化，是解决大规模数据处理问题的关键。通过不断学习和实践，我们能够更加熟练地运用这些算法工具，解决更为复杂的实际问题。

参考文献

- [1] KONNYAKUXZY. OI Wiki - 莫队算法[EB/OL]. (2018-07-30). <https://oi-wiki.org/misc/mo-algo/>.
- [2] OOJ_BAI. 重新编号颜色优化莫队算法[EB/OL]. (2024-09-19). <https://www.luogu.com.cn/discuss/929927>.
- [3] PHANTOM_LANDLER. 通过优化桶优化莫队算法[EB/OL]. (2024-11-25). <https://www.luogu.com.cn/discuss/1005310>.

致 谢

- 感谢「沪疆教育信息化 算法小论文」提供的学习和交流的平台。
- 感谢张卫国老师、诸峰老师对我的指导和帮助。
- 特别感谢程佳润同学对我的支持。
- 感谢洛谷平台提供的题目和测试。